



Harnessing Inter-GPU Shared Memory for Seamless MoE Communication-Computation Fusion

Hulin Wang
wonghulin@whu.edu.cn
School of Computer Science
Wuhan University
Wuhan, China

Yaqi Xia
yaqixia@whu.edu.cn
School of Computer Science
Wuhan University
Wuhan, China

Donglin Yang
dongliny@nvidia.com
Nvidia Corporation
Santa Clara, USA

Xiaobo Zhou
waynexzhou@um.edu.mo
IOTSC
University of Macau
Macau S.A.R

Dazhao Cheng*
dcheng@whu.edu.cn
School of Computer Science
Wuhan University
Wuhan, China

Abstract

The Mixture of Experts (MoE) architecture enhances model quality by scaling up model parameters. However, its development is hindered in distributed training scenarios due to significant communication overhead and expert load imbalance. Existing methods, which only allow for coarse-grained overlapping of communication and computation, slightly alleviate communication costs but at the same time, they introduce a notable impairment of computational efficiency. Furthermore, current approaches to addressing load imbalance often compromise model quality.

We introduce CCFuser, a novel framework designed for efficient training of MoE models. CCFuser replaces the costly All2All operations typical in MoE architectures with efficient inter-GPU shared memory access. This allows for the concurrent computation of local and remote data within a fused kernel, achieving substantially higher compute FLOPS for GEMM operations. Additionally, CCFuser addresses load imbalance with a resource-efficient expert reassignment strategy, which optimizes the use of computational resources in expert reassignment through equivalent graph transformations without sacrificing statistical accuracy. By integrating these optimizations, CCFuser significantly enhances GPU utilization efficiency. Experiments conducted on A100 servers

show that CCFuser outperforms state-of-the-art methods FastMoE and FasterMoE by an average of 2.96x and 2.48x, respectively, achieving a maximum speedup of 4.34x.

CCS Concepts: • Computing methodologies → Parallel algorithms.

Keywords: MoE, Kernel Fusion, GPU

ACM Reference Format:

Hulin Wang, Yaqi Xia, Donglin Yang, Xiaobo Zhou, and Dazhao Cheng. 2025. Harnessing Inter-GPU Shared Memory for Seamless MoE Communication-Computation Fusion. In *The 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '25)*, March 1–5, 2025, Las Vegas, NV, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3710848.3710868>

1 Introduction

Large language models have achieved significant breakthroughs in fields such as natural language processing [1, 6] and computer vision [2, 36]. These achievements demonstrate that increasing model size can enhance model quality. However, despite their effectiveness, the training of these large models incurs extremely high computational costs. For example, training Meta’s Llama2-70B model [33] requires 3 million GPU hours on A100-80GB GPUs.

A straightforward method to reduce the computational cost of large-scale models is through sparse activation [10]. The Mixture of Experts (MoE) architecture, well-documented for achieving sparse activation, has proven effective in various studies [16, 27]. Unlike dense models that compute all parameters, MoE selectively activates only a subset, allowing for substantial model size increases without a proportionate rise in computational demands. For instance, the MoE-based Mixtral-8x7B model [16] contains 46.7 billion parameters, yet only 12.9 billion parameters are activated during computations. This model achieves performance comparable to Llama2-70B while operating six times faster. To manage the

*Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. PPoPP '25, March 1–5, 2025, Las Vegas, NV, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1443-6/25/03...\$15.00

<https://doi.org/10.1145/3710848.3710868>

large parameter sizes of MoE models, current training systems [14, 17, 18] support distributed training by assigning experts across multiple GPUs, a method known as expert parallelism. While the method mitigates high memory demands of MoE models, it adversely affects training efficiency.

MoE’s expert parallelism relies on All2All operations facilitated by the NVIDIA NCCL library for token distribution and collection. During these operations, computational resources in worker nodes often remain idle. A recent study [14] has shown that over 40% of training time in MoE setups is consumed by All2All communication, significantly affecting performance. To mitigate this issue, several approaches [12, 19, 29, 30, 42] have been developed to accelerate training through inter-operator scheduling. This strategy involves dividing input data into smaller batches and designing scheduling algorithms to overlap communication and computation operations (GEneral Matrix-to-matrix Multiply, e.g., GEMM), thus minimizing communication overhead. Despite these advancements, these approaches still struggle to eliminate GPU idle time during the initial and final phases of communication. Moreover, finer partitioning of data inputs, while reducing the communication load, leads to smaller computation chunks and decreased GPU utilization efficiency.

Another factor that affects the efficiency of MoE training is the expert load imbalance issue. In MoE, input data (tokens) are distributed to various experts via a dynamic routing network, resulting in an uneven number of tokens processed by each expert and an imbalanced load across the GPUs. Strategies [10, 12, 17] address this by capping the number of tokens each expert can handle. However, these methods can lead to tokens being routed to sub-optimal experts or even dropped, compromising model quality. Other approaches [23] strive to balance loads among GPUs by reassigning experts. While effective in achieving balance, these methods still necessitate significant token transfers between GPUs, leading to excessive communication overhead. Furthermore, the algorithms for expert reassignment primarily utilize communication resources, resulting in under-utilization of computational resources and poor GPU efficiency.

To address the limitations of existing MoE training frameworks, we introduce CCFuser, a novel framework designed to significantly enhance training efficiency and load balance. CCFuser features two key innovations: 1) a GEMM kernel fusion that leverages inter-GPU shared memory, effectively eliminating the communication overhead associated with All2All operations in MoE model training; and 2) a resource-efficient expert reassignment strategy that optimizes load balancing across GPUs while improving data locality, thereby enhancing GPU utilization.

Firstly, instead of the traditional coarse-grained inter-operator overlap, we propose to seamlessly fuse communication and computation operations into a single, optimized GEMM kernel utilizing inter-GPU shared memory. This fusion strategically avoids the idling of GPU computational

resources typically caused by standalone communication operations. By simultaneously overlapping communication and computation at both inter- and intra-threadblock levels within the kernel, we effectively conceal the overhead associated with accessing remote data, thereby maximizing GPU computational resource utilization. To facilitate this seamless fusion, we organize data meant for the All2All operation within the inter-GPU shared memory and design a new data structure incorporating a tiling scheme. Additionally, by integrating all local GPU expert computations into the GEMM kernel, we significantly enhance MoE expert computation efficiency and address one of the primary bottlenecks in MoE training — GPU utilization.

Secondly, we achieve load balancing across GPUs through a resource-efficient expert reassignment algorithm. By transforming the structure of the MoE layer, we enable simultaneous execution of the expert assignment algorithm and attention layer computation, maximizing communication and computation resource utilization. Depending on the token distribution, we employ distinct assignment strategies for popular and unpopular experts to enhance data locality within each GPU. For popular experts, we utilize expert replication to improve data localization. Conversely, for unpopular experts, we facilitate expert reassignment across GPUs, effectively alleviating memory pressure.

In summary, our key contributions are as follows:

1. We introduce a novel fused kernel based on inter-GPU shared memory, which is capable of processing both local and remote data simultaneously within the kernel. This innovation completely eliminates the cross-GPU synchronization required for All2All operations.
2. We develop a strategy to overlap communication and computation at both inter- and intra-threadblock levels, effectively concealing the overhead of accessing remote data and maximizing GPU computational resource utilization.
3. We propose a resource-efficient expert reassignment algorithm that significantly enhances data locality and balances workloads. We reduce algorithmic overhead by transforming the MoE structure, leading to substantially improved computational resource utilization.

We integrated CCFuser into PyTorch for MoE model training. Tested on NVIDIA A100 servers, CCFuser achieved substantial performance gains over FastMoE and FasterMoE, with average speedups of 2.96x and 2.48x, respectively, and a maximum speedup of up to 4.34x.

2 Background and motivation

2.1 MoE Structure and Expert Parallelism

The gated MoE architecture [28] consists of multiple sub-models known as experts, which are typically implemented

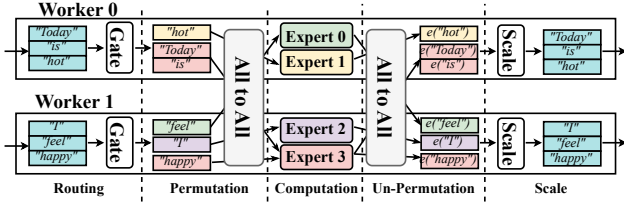


Figure 1. A MoE Layer is depicted in its generalized form, where *expert_num*=4, *top_k*=1, *worker_num*=2. Experts 0-1 reside on worker0, while experts 2-3 are on worker1.

as feed-forward neural network (FFN) [10]. Experts are dynamically selected by routing to process input, making the model more efficient and expressive [10, 27].

In Transformer models, the MoE layer often replaces the FFN layer, increasing parameters and performance without significantly raising computational costs. When parameters exceed a single GPU’s capacity, expert parallelism distributes MoE experts across multiple GPUs. The MoE layer’s structure, shown in Figure 1, includes five phases: Routing, Permutation, Computation, Un-permutation, and Scaling.

Routing: The MoE layer’s first stage assigns tokens to experts. Tokens pass through a Gate network, producing a token-expert score matrix. After softmax normalization, the top k experts are selected for each token.

Permutation: After determining the expert indices for all input tokens, the Permutation phase scatters token vectors within each worker to consolidate them for the same experts. An All2All operation then distributes tokens across workers to their respective experts for processing.

Computation: Once the data has been permuted, the experts perform calculations on their assigned inputs to produce the results.

Un-permutation: After expert computation, an All2All operation redistributes the result vectors to the original workers, followed by a local gather to reorder the vectors according to their original MoE input arrangement.

Scaling: The final MoE step scales output tokens by their expert scores, summing weighted results for tokens assigned to multiple experts to produce the final layer output.

2.2 NVIDIA Communication Library

NVIDIA provides two primary communication libraries for multi-GPU environments: NCCL [24] and NVSHMEM [25].

NCCL is a high-performance communication library developed by NVIDIA, tailored specifically for deep learning and high performance computing environments utilizing multiple GPUs. It offers optimized collective communication operations such as All-Reduce, Broadcast, and All-Gather. Commonly integrated with popular deep learning frameworks like TensorFlow and PyTorch, NCCL facilitates efficient data synchronization across GPUs.

NVSHMEM, in contrast, is a shared memory communication library designed for NVIDIA GPUs and has been extensively utilized in research related to Graph Neural Networks [37, 38]. Inspired by the traditional shared memory model (SHMEM), it partitions GPU memory into inter-GPU shared and private segments. This setup enables processes on one GPU to access data on another GPU via NVSHMEM shared memory addresses and associated GPU IDs. The library provides various APIs for synchronization and communication, catering to diverse programming models and application scenarios. Unlike NCCL, which focuses on comprehensive communication operations, NVSHMEM is more composable, offering a lower-level API that enables more granular data access within kernels.

2.3 Motivations

2.3.1 Enhancing MoE Efficiency: Shifting from NCCL to NVSHMEM. For training MoE models, traditional execution of expert parallelism requires the All2All communication during both the permutation and un-permutation phases, resulting in significant communication overhead.

We conduct an experiment on an MoE layer with 64 experts using a server equipped with four A100 PCIe GPUs. Experimental results reveal that 41% of the total processing time is consumed by All2All communication. This prolonged duration of All2All communication is identified as the primary source of inefficiency in the MoE model training.

The All2All communication heavily utilizes network resources. The inherent data dependencies between the permutation, computation, and un-permutation phases in the MoE model result in idle GPU computational resources during these communication periods, as shown in Figure 2(a). The longer the duration of All2All communication, the longer the GPUs remain idle, thereby reducing overall efficiency. Existing approaches [12, 19, 21, 29, 30] recommend partitioning the input data into smaller chunks (small tensors) and strategically scheduling the communication and computation sub-tasks of these chunks. These strategies aim to maximize the overlap between computation and communication operations. As illustrated in Figure 2(b), dividing the input data into two chunks and subsequently scheduling their sub-tasks can effectively overlap the expert computations of the chunks with the All2All communications.

However, these approaches rely on coarse-grained communication with NCCL, encountering two major issues: 1) It is impossible to completely eliminate idle GPU states during All2All communication, as depicted in the *All2All 0-0* and *All2All 1-1* scenarios in Figure 2(b). 2) Since the data is divided into smaller tensors, these are unable to fully utilize the GPU’s computational capacity. Consequently, the total time cost after partitioning, shown as T_{e2} in Figure 2(b), is longer than the naive time cost T_{e1} in Figure 2(a).

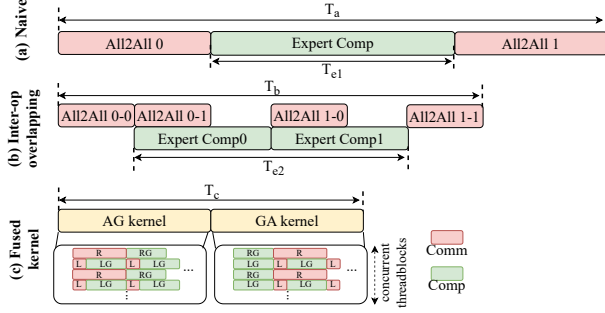


Figure 2. Subtask scheduling for MoE. (a) Traditional execution. (b) Scheduling tasks in data chunks. (c) Fused kernel.

Compared to NCCL, NVSHMEM offers finer-grained memory access, which in turn provides an opportunity to seamlessly fuse communication and computation within the kernel. By leveraging NVSHMEM for All2All functionality instead of NCCL, GPU idling can be significantly reduced, as illustrated in Figure 2(c). However, employing NVSHMEM to achieve seamless MoE communication-computation fusion presents two significant challenges.

(1) **Complex Data Management.** The dynamic nature of MoE’s network routing means that the data processed by experts continually changes, complicating the task for the fused kernel to accurately retrieve the correct data for computation from inter-GPU shared memory. Moreover, as kernels process both local and remote data, traditional GEMM kernel data tiling along both row and column dimensions can degrade performance due to frequent remote accesses. Consequently, we need to develop a suitable data structure and tiling scheme that effectively manage data and enhance the performance of fused kernel.

(2) **High Overhead of Remote Data Access.** Although NVSHMEM offers finer-grained memory access than NCCL, it does not inherently reduce the overhead associated with remote data access. When threadblocks require access to remote data, the ensuing high communication costs can cause operations to be blocked, wasting computational resources and extending kernel execution times. Therefore, our aim is to fully utilize idle GPU cycles caused by remote data access to conceal this overhead, thereby maximizing the utilization of GPU hardware processing units.

2.3.2 Load Balancing in MoE Computation. Due to dynamic routing in MoE models, input tokens are distributed to different experts for computation. To investigate the load balancing issue, we replaced the FFN layer in the Transformer-XL model [5] with an MoE layer consisting of 64 experts. These experts are distributed across four A100-PCIE GPUs, with each GPU handling 16 experts. After training on the enwik8 [13] dataset, the resulting metrics are displayed in Figure 3. The data reveals a significant uneven preference of input tokens for certain experts, leading to a severe imbalance in workload distribution among the experts, as depicted

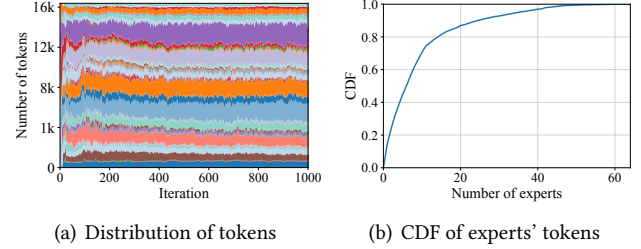


Figure 3. Distribution of tokens in MoE training.

in Figure 3(a). This imbalance extends to the GPUs themselves, resulting in sub-optimal resource utilization.

Several existing methods [12, 17] attempt to achieve load balancing on GPUs by limiting the number of tokens each expert processes. However, these approaches often lead to a significant number of tokens being reassigned to sub-optimal experts—or even dropped, ultimately compromising the model’s quality. Unlike previous methods that compromise model quality to achieve load balancing, FlexMoE [23] offers a direct approach to balancing the workload across GPUs by reassigning experts. However, this approach introduces two non-trivial challenges:

(1) **Poor GPU Data Localization.** The MoE layer’s experts are categorized into popular and non-popular groups. The top-6 popular experts (out of 64) handle more than 50% of the input tokens, as shown in Figure 3(b). Without considering expert popularity in assignments, ideally, the percentage of local data processed per GPU would be about $1/N$ (where N is the number of GPUs). This arrangement results in a significant number of tokens being transferred to other GPUs for processing, leading to poor data locality and reduced GPU computational efficiency.

(2) **Waste of GPU Computational Resources.** The expert reassignment algorithm operates between the gate network and expert computation within the MoE layer. Due to the weight data dependencies from the expert computation, the expert computation is delayed, impacting training efficiency. Moreover, as the expert reassignment algorithm requires the exchange of expert weights between GPUs, it consumes inter-GPU communication resources, leading to wasted computational resources.

3 CCFuser Design

We introduce CCFuser, a novel approach that seamlessly fuses computation and communication for implementing MoE models using inter-GPU shared memory. CCFuser leverages NVSHMEM to enable cooperative sharing and access of MoE input tokens across GPUs. This framework achieves fine-grained overlap of communication and computation at both inter-threadblock and intra-threadblock levels, thereby maximizing GPU computational resource utilization.

CCFuser introduces two major innovations:

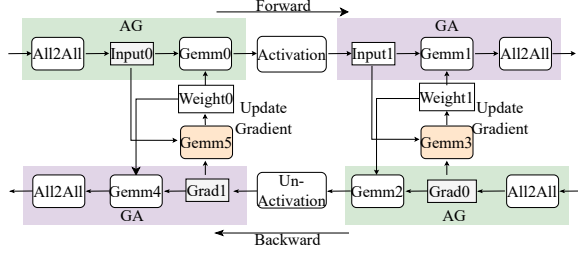


Figure 4. The data flow of the MoE forward and backward.

1. **Inter-GPU Shared Memory based GEMM:** This feature replaces traditional All2All communication with inter-GPU shared memory access to implement a fused GEMM kernel. By overlapping communication and computation hierarchically, it significantly enhances MoE computation efficiency.
2. **Resource-efficient Expert Reassignment:** This strategy reassigns experts based on their popularity across GPUs, ensuring balanced load distribution while improving data locality, thus boosting overall GPU communication and computational resource efficiency.

3.1 Inter-GPU Memory Sharing Based GEMM

In the expert parallelism of MoE models, a significant amount of time is spent on All2All operations during the permutation and un-permutation stages, resulting in idle GPU compute resources. To enhance GPU utilization, we have designed a novel GEMM kernel that utilizes inter-GPU shared memory. This new kernel significantly accelerates computation by effectively overlapping communication and computation at both inter-threadblock and intra-threadblock levels.

3.1.1 AG and GA Kernels. Consider a MoE FFN layer where each expert is a 2-layer MLP, as illustrated in Figure 4. In the forward pass, an All2All operation transfers data from various workers to their respective expert workers before executing the GEMM operation for the first MLP. After completing the GEMM operation for the second MLP, another All2All operation sends the token back to its original worker. During the backward pass, this process is reversed, and each MLP performs an additional GEMM operation to compute the gradients of the weights.

Traditionally, the dependency between All2All operations and the adjacent GEMM operations results in GPU compute resources remaining idle during All2All communication. To address this inefficiency, we have integrated the NVSHMEM library to fuse the forward and backward GEMM operations with their corresponding All2All operations into a unified kernel within the MoE architecture. This integration enables the kernel to process local and remote data simultaneously, thereby eliminating the GPU idle time associated with NCCL-based All2All operations. Consequently, we have developed two innovative kernels: the All2All-GEMM

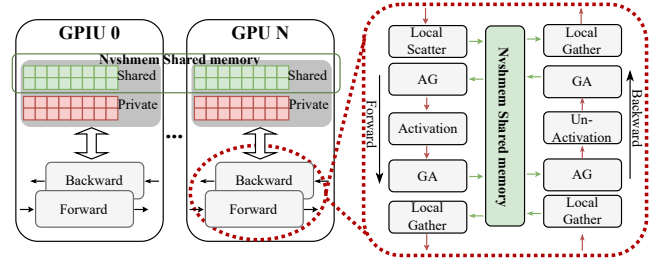


Figure 5. Interactions between MoE operations and inter-GPU shared memory. The red box indicates interactions during forward and backward. Arrows showing data flow.

(AG) kernel, which manages both the All2All and the first GEMM operations during forward and backward passes, and the GEMM-All2All (GA) kernel, which oversees the second GEMM and All2All operations. This strategy of mixing local and remote workloads within a single kernel significantly improves GPU utilization, optimizing both the computational processes and resource efficiency.

How can NVSHMEM be used to replace All2All operations in computations? In the MoE architecture, All2All operations are essential before the first MLP and after the second MLP. To substitute NCCL-based All2All with NVSHMEM, we utilize NVSHMEM shared memory to efficiently store the first MLP’s input and the second MLP’s output. To achieve this, we partition the global GPU memory into inter-GPU shared memory and private memory. The inter-GPU shared memory stores the MoE inputs and outputs, making them accessible by all GPUs, while the private memory holds weights, activations, and other essential tensors.

This setup facilitates the interaction between MoE operations and inter-GPU shared memory, as depicted in Figure 5. Before executing the AG operation, we write the results of the *local scatter* to the inter-GPU shared memory. During the AG kernel, the NVSHMEM remote data access API *nvshmemx_getSIZE_nbi_block* retrieves the MoE layer’s input data for GEMM computation. Subsequently, in the GA kernel, after completing the GEMM computation, the results are written back to the inter-GPU shared memory using *nvshmemx_putSIZE_nbi_block*. Lastly, each GPU uses the *local gather* operation to reorder the data from the local inter-GPU shared memory according to the MoE inputs. The backward operation mirrors the forward operation.

By adopting a fused kernel based on inter-GPU shared memory, we completely eliminate the cross-GPU synchronization operations typically required by All2All operations in other approaches, thereby significantly enhancing GPU utilization efficiency.

3.1.2 Data Format and Data Tiling. To efficiently map data to the GPU’s tensor core units, we divide the data into blocks along the row dimension and store the source data for these blocks in NVSHMEM shared memory. Upon kernel

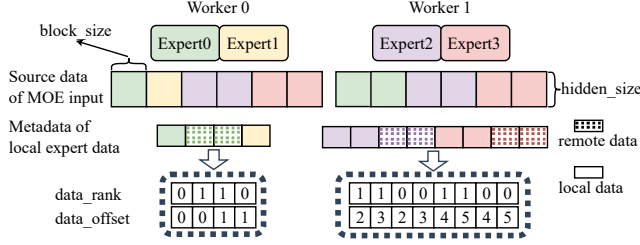


Figure 6. Source data and metadata for MoE.

launch, each threadblock identifies the specific NVSHMEM shared memory addresses it needs to access. As illustrated in Figure 6, each worker stores metadata pertinent to the data required by the local expert. In the expert computation of the 2-layer MLP, the number and sequence of rows in the input and output matrices are constant. Consequently, this metadata can be consistently shared between the forward and backward AG kernels and the GA kernel.

The metadata variables *data_rank* and *data_offset* denote the GPU index and the offset within the NVSHMEM shared memory, respectively. These variables are crucial for the AG and GA kernels to accurately and efficiently locate expert data within the shared memory. Given that we only need to store a set of metadata of size $block_size \times hide_size$, the memory requirement for this additional metadata is minimal, optimizing system resource use.

How can we achieve efficient data tiling between threadblocks? Figure 7(a) illustrates the common method of data tiling for GEMM computation, where the output matrix is divided into submatrices along both row and column dimensions, with each submatrix assigned to a threadblock for computation. However, this traditional data tiling approach is not suitable for AG and GA kernels due to inefficiencies related to remote data access. In these kernels, accessing data on remote GPUs can be tens to hundreds of times slower than accessing local GPU memory. As demonstrated in Figure 7(a), when computing the shaded portions, four threadblocks may need to access the same remote data, leading to decreased kernel performance.

For our AG and GA kernels, we have developed a data tiling strategy that divides the output matrix along the row dimension only, as depicted in Figure 7(b). This method ensures that all remote data is accessed only once during the computation, significantly reducing the number of low-bandwidth data accesses and enhancing overall GPU utilization.

With new data tiling, remote and local data blocks can be assigned to different threadblocks within the kernel, enabling the GPU’s SMs to process both types of data simultaneously. The scheduling of these threadblocks is managed by the GPU scheduler, which optimizes resource use: while one threadblock is accessing remote data, others can concurrently compute local data. This strategy of inter-threadblock

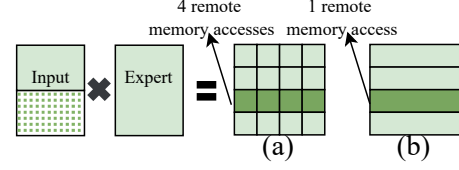


Figure 7. Data tiling for sparse GEMM, which is divided by (a) row and column dimensions, and (b) row dimension.

overlapping communication and computation across different SMs significantly enhances overall efficiency.

3.1.3 Fine-grained Intra-threadblock Overlapping. As previously discussed, inter-threadblock overlapping can enhance GPU occupancy. However, it does not fully maximize the GPU’s computational resources. When threadblocks access remote data blocks, threads often experience significant delays while waiting for the data, which in turn prolongs the overall kernel runtime.

To address this issue, we have the threadblocks handle both remote and local data, enabling communication latency to be overlapped, thereby improving GPU resource utilization.

In Figure 8, the left side illustrates the sequential processing of data blocks within a threadblock. It shows that when accessing remote *block 1* and *block 2*, the threadblock must wait for the completion of remote data access, resulting in a blocked state. To minimize the time a threadblock is blocked due to remote data access, we propose an adaptive overlap method for communication and computation within threadblocks, as depicted on the right side of Figure 8. In the AG kernel, we first use asynchronous read APIs to access remote data blocks, followed by local data computation, and then computation on the remote data blocks. Conversely, in the GA kernel, computation on remote data blocks is performed first, after which the results are asynchronously written back, and finally, local data computation is conducted. This approach allows the SMs to utilize compute resources for local data computation while concurrently managing read and write operations for remote data, thereby maximizing GPU compute resource utilization.

Since NVSHMEM’s remote data access operates on a peer-to-peer (P2P) model, its speed depends on the bandwidth rate between the two GPUs involved. As illustrated in Figure 8, *block 1* is connected via NVLINK, while *block 2* is connected via a PCIe bridge. Consequently, *block 1* experiences shorter access times compared to *block 2*. Therefore, it is crucial to select the appropriate number of local data blocks to overlap with remote data blocks, based on the bandwidth connection between the remote and local GPUs. For instance, as depicted in Figure 8, we overlap one local data block with remote *block 1* and two local data blocks with remote *block 2* to balance the data processing speeds efficiently.

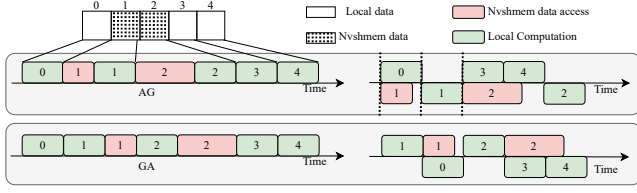


Figure 8. Adaptive overlapping between communication and computation within the threaded block.

3.1.4 Multi-Expert Fusion. In the default MoE layer, each expert processes the assigned input tokens independently. However, launching multiple kernels introduces additional system overhead. Moreover, uneven token distribution can lead to inefficiencies, as underutilized experts handle too few tokens to fully utilize the GPU. Horizontally fusing multiple expert kernels into a single one can address both issues.

When fusing multiple expert kernels, each thread block in the AG and GA kernels needs to retrieve the required expert weights to process the input tokens. It is necessary to introduce the metadata, i.e., *data_expert_id* to help locate each data block. For instance, the *data_expert_id* for worker 0 might be $0, 0, 0, 1$. Within the kernel, thread blocks access local or remote data according to the metadata’s *data_rank* and *data_offset*, using *data_expert_id* to locate the weights of local experts, facilitating the fused kernel computation.

In the backward pass of the MoE for computing weight gradients, fusing multiple expert kernels transforms the computation from a two-dimensional to a three-dimensional problem. Managing the data blocks during this computation poses a challenge. To address this, we propose tiling the output gradients along three dimensions: the number of experts, and the rows and columns of the weight matrix. This data tiling strategy ensures enhanced parallelism. Besides, different from AG and GA, which perform GEMM on data and weights, gradient computation GEMM takes the forward pass *Activation* and backward pass *Grad* as inputs. Therefore, an additional metadata, *expert_data_offset*, is required to index the correct *Activation* and *Grad* for the fused GEMM kernel thread blocks. As shown in Figure 6, worker 0’s *expert_data_offset* is $\{0, 3, 4\}$. This means the first expert’s data spans blocks 0-2, and the second expert’s data is block 3.

With the Horizontal fusion, all experts in the MoE layer are fused into a single kernel in both MoE forward and backward passes, greatly accelerating the computation.

3.2 Resource-efficient Expert Reassignment

The distribution of tokens by MoE experts is often uneven, leading to a significant number of tokens being sent to other GPUs for computation. As the number of tokens sent increases, the communication overhead grows, resulting in poor GPU utilization and reduced efficiency. Our expert reassignment approach improves GPU utilization by enhancing

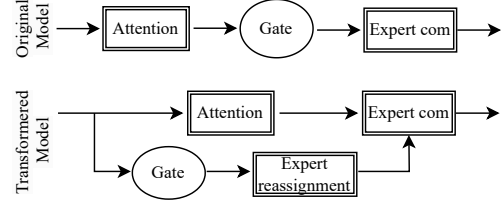


Figure 9. Transformation of an MoE layer.

data locality based on the token distribution. Specifically, we modify the MoE layer and pre-run the gate network to determine the token distribution. Then, we reassign experts among GPUs to improve data localization, ensure workload balance across GPUs without sacrificing statistical accuracy, and parallelize expert reassignment with operations that are free from data dependencies to optimize GPU communication and computational resource efficiency.

3.2.1 Transformation of MoE Layer. If the expert reassignment algorithm is executed before the MoE computation, the MoE computation must wait for the reassignment algorithm to complete, leading to decreased training efficiency. To address this issue, we transform the structure of the MoE layer by simultaneously performing the computation of the gate network and attention, as shown in Figure 9. The input for the attention module also serves as the input for the gate network, allowing us to select the appropriate expert for each token. With the pre-determined token routing information, we can execute the expert reassignment algorithm in advance. It is important to note that there is no data dependency between the expert reassignment algorithm and the attention mechanism. And, the expert reassignment primarily consumes inter-GPU bandwidth, whereas the attention computation utilizes the computational resources of the GPUs. Consequently, the expert reassignment algorithm is processed in parallel with the attention layer computation.

This transformation of the MoE makes sense because the gate network typically employs a simple model architecture where routing or expert selection is often determined by straightforward features, such as the syntactic roles (e.g., nouns, verbs) or semantic meanings (e.g., time, numbers) of tokens [18]. These features are fixed for each token. Additionally, due to the simplicity of the gating network, it can capture the syntactic or semantic properties of token even when using feature vectors from the previous layer during training. Based on the above reasons, the transformation of the MoE is effective, and we have validated in Section 4.5 that it does not affect the model’s convergence speed.

3.2.2 Expert Reassignment. With the token distributions on different GPUs, we can implement our expert reassignment algorithm to enhance data localization on GPUs. The algorithm is structured as follows: in the MoE layer, given a distribution of tokens $\mathcal{N}$ on different GPUs, we need to assign all the experts in set $\mathcal{E}$ to GPUs on the multi-GPU

platform $\mathcal{G}$, considering the bandwidth $\mathcal{B}$ between GPUs, to determine the final expert assignment plan $\mathcal{P}$. Our objective is to maximize data locality on GPUs, calculated as:

$$\mathcal{L}(g', \mathcal{P}) = \frac{\sum_{e^{(e,g')} \in \mathcal{P}} \mathcal{N}_{e,g'}}{\sum_{e \in \mathcal{E}} \mathcal{N}_{e,g'}} \quad (1)$$

The experts in the MoE are categorized into popular and unpopular experts, necessitating different assignment strategies for each type. For popular experts, it is more efficient to keep their computations localized since they handle a large proportion of input data tokens. For unpopular experts, which process fewer tokens, we distribute them across multiple GPUs to alleviate memory pressure.

We identify the popular experts on GPU g' by analyzing the token distribution $\mathcal{N}_{e,g'}$ of the experts on that GPU. Consequently, we sort $\mathcal{N}_{e,g'}$ in descending order to select the smallest set $\mathcal{E}_p$ of the popular experts on this GPU, which meet the following criteria.

$$\mathcal{E}_p = \arg \min_{\mathcal{E}_p} \{ |\mathcal{E}_p| \mid \frac{\sum_{e \in \mathcal{E}_p} \mathcal{N}_{e,g'}}{\sum_{e \in \mathcal{E}} \mathcal{N}_{e,g'}} > 50\% \} \quad (2)$$

Where $|\mathcal{E}|$ denotes the number of elements in the set $\mathcal{E}$. We require that the total number of tokens from popular experts account for more than 50% of the local tokens, as at least 50% of local data is needed to address the overlap. After obtaining $\mathcal{E}_p$, we add the tuple $\{e', g'\}$ to the expert reassignment plan $\mathcal{P}$, where $e' \in \mathcal{E}_p$.

For unpopular experts, we need to assign them to different GPUs. Since the kernel implementation is based on NVSHMEM and follows a P2P communication model, their access speeds are directly related to the communication bandwidth between GPUs. Therefore, when assigning unpopular experts, we need to consider the network topology between GPUs, maximizing bandwidth resource utilization. For the assignment of a specific expert e' , we should follow the criteria below to select the appropriate GPU g' .

$$g' = \arg \max_{g'} \left\{ \sum_g \mathcal{N}_{e',g} \times \mathcal{B}_{g,g'} \right\}, \text{ where } g' \in G \quad (3)$$

Then, add the tuple $\{e', g'\}$ to the assignment plan $\mathcal{P}$.

In the expert reassignment algorithm, we first compute the data locality of the previous assignment plan $\mathcal{P}$ using Equation 1. If $\mathcal{P}$ meets the minimum data locality threshold l_{lb} , it is retained. Otherwise, we generate a new plan, $\mathcal{P}'$. This begins by selecting popular experts for each GPU according to Equation 2, integrating them into $\mathcal{P}'$ with their $\mathcal{N}_{e,g}$ values set to 0 to avoid conflicts in the subsequent steps. Next, we assign unpopular experts to appropriate GPUs using Equation 3 and add them to $\mathcal{P}'$. The process concludes with the assignment of all experts according to $\mathcal{P}'$, resulting in an updated assignment plan.

With the expert assignment algorithm, we can significantly enhance data locality on GPUs. This addresses load

imbalance and improves kernel computational efficiency. Due to the iterative nature of model training, the assigned experts on each GPU remain mostly unchanged, leading to minimal overhead from occasional expert reassignments. Consequently, the additional overhead introduced by our method during training is minimal.

3.3 Implementation

Built on CUDA 11.7 and PyTorch 2.0.1, CCFuser is seamlessly integrated into the PyTorch ecosystem through PyTorch Extensions. To optimize performance, CCFuser employs expert parallelism by distributing experts across multiple GPUs while applying data parallelism to other model components. It provides a user-friendly Python API, enabling developers to efficiently implement custom MoE models. CCFuser is compiled and linked using PyTorch (v2.0.1), CUDA (v11.7), OpenMPI (v4.1.1), NVSHMEM (v2.9.0), and cuDNN (v8.2). The open-source artifact has been available on GitHub: <https://github.com/WongHuLin/CCFuser.git>.

4 EVALUATION

4.1 Experimental settings

Testbed. We evaluate CCFuser on two servers. The first server, *pinnacle*, is equipped with two Intel 8269CY processors (26 cores, 52 threads, 2.50 GHz each). It includes 256 GB of system memory and four NVIDIA A100 PCIe GPUs (40 GB memory each), connected via NVLink bridges and PCIe buses. This server represents a typical deep learning training workstation. The second server, *vertex*, features two AMD EPYC 7543 processors (32 cores per processor, 2.8 GHz) and four NVIDIA® Tesla A100 GPUs (40 GB memory each). These GPUs are interconnected through NVLink in a fully connected topology, offering up to 600 GB/s of GPU-to-GPU communication bandwidth. This configuration is ideal for high-performance computing.

Approaches. We compare CCFuser with two frameworks for MoE training optimization. FastMoE [11] is an MoE training system based on PyTorch and general-purpose accelerators, while FasterMoE [12] builds on FastMoE by scheduling communication and computation operations to minimize communication overhead during MoE training.

Metrics. We evaluate CCFuser using three key metrics:

- MoE Layer Time: for one MoE layer of computation and communication.
- Training Step Time: to complete one training step.
- Kernel Execution Time: to complete the fused kernel, either AG or GA.

To collect these metrics, we use CUDA Events to measure the execution time of CUDA kernels and communication. The results are averaged over 80 steps after 20 warm-up runs.

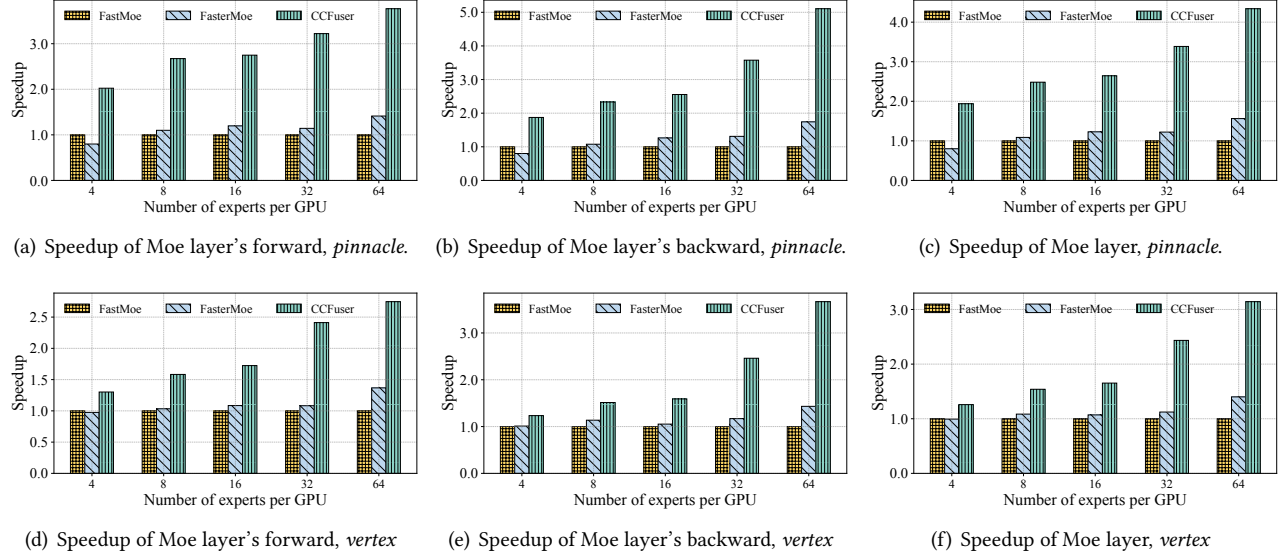


Figure 10. Training time speedup over FastMoE of MoE layer with different number of experts.

4.2 Performance of the MoE Layer

We first evaluate the performance of MoE layers on two platforms. The evaluation method involves increasing the number of experts per GPU and measuring the MoE forward, backward, and total time during the experiment using identical input configurations across all frameworks (batch size = 16, seq len = 512, top_k = 2, model dim = 1024). Performance of FastMoE is used as a baseline for the results.

Figure 10 illustrates the training time acceleration of MoE layers across different configurations. We specifically seek to understand the performance gains provided by CCFuser in both the forward and backward of the MoE layers. As shown in Figures 10(a), 10(b), 10(d) and 10(e), the gains from CCFuser increase with the number of experts, resulting in an average performance improvement of 2.42x in the forward and 2.59x in the backward over FastMoE. Compared to the forward, the backward in MoE layers involves two additional GEMM operations for calculating weight gradients, which are also optimized by CCFuser. As a result, the performance improvement in the backward pass is more significant.

Regarding the overall time for MoE layers, as depicted in Figures 10(c) and 10(f), CCFuser outperforms FastMoE and FasterMoE, achieving average performance improvements of 2.96x and 2.48x on *pinnacle*, and 2.0x and 1.73x on *vertex*, respectively. The performance boost is more pronounced on *pinnacle* compared to *vertex*, primarily due to *vertex* GPUs being interconnected via NVLink, which offers higher communication bandwidth. This reduces communication overhead in the MoE layers and limits the optimization potential of our CCFuser method.

Compared to FastMoE, FasterMoE can overlap computation and communication operations for the experts, thereby

Table 1. Configuration of models.

model	Batch size	Seq len	Model dim	Moe block	Total block	Expert num
M-GPT	8	1024	768	1	12	64
M-BERT	32	512	768	4	12	64
M-Trans-xl	16	512	512	12	12	64

hiding communication overhead. However, its efficiency varies with the number of experts. When the number of experts is low, FasterMoE may even perform worse than FastMoE, as shown in Figure 10(c) and Figure 10(f), where the number of experts is 4. In contrast, our implementation seamlessly fuses MoE communication and computation into CUDA kernels, eliminating GPU idle time caused by communication. This leads to significant speedups in our experiments, especially as the number of experts increases, where the communication overhead becomes more pronounced, resulting in even greater optimization benefits.

4.3 End-to-end performance of the MoE model

We evaluated CCFuser using three models: MoE-GPT, MoE-BERT, and MoE-Transformer-xl. BERT is a representative model based on the Transformer encoder, while MoE-GPT and MoE-Transformer-xl are representative models based on the Transformer decoder. The configurations of these models are shown in Table 1. All these models consist of 12 Transformer blocks. For the BERT and GPT models, we replaced certain blocks with MoE blocks: specifically, 2nd, 5th, 8th, and 11th blocks in the BERT model, and 11th block in the GPT model. In the Transformer-xl model, all blocks are replaced with MoE blocks.

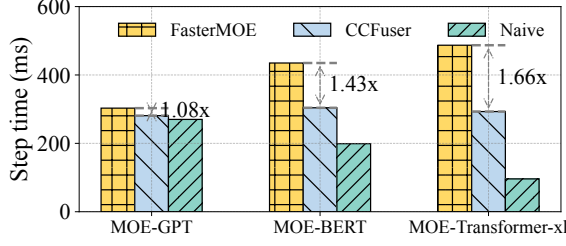


Figure 11. Performance of CCFuser, Naive, and FasterMoE.

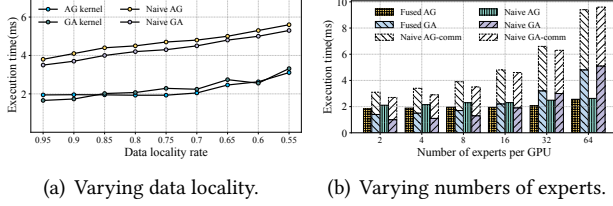


Figure 12. Impact of fused kernel on performance.

We compared the performance of FasterMoE, Naive (without MoE layer), and CCFuser across three models, using the training step time as an indicator of system efficiency. Figure 11 shows the performance comparison between FasterMoE and CCFuser. Replacing the original model’s FFN layer with MoE layer introduces additional communication and computation overhead, resulting in higher step times for both FasterMoE and CCFuser compared to Naive. However, our approach significantly reduces this overhead, achieving speedup of 1.08x, 1.43x, and 1.66x across the three models compared to FasterMoE. Nonetheless, the overall end-to-end performance improvement is slightly less impressive compared to the speedup in the MoE layer. This is because the time cost per iteration includes not only the communication and computation of the MoE layer but also other model components, such as attention modules and embedding layers. Since these components are not optimized in our approach, the overall performance improvement is compromised.

4.4 Analysis of the Fused Kernel

The key contribution in this paper is fusing MoE communication and computation into CUDA kernels. To evaluate the performance of the fused kernels, we test them from two perspectives: data locality and the number of experts.

Data locality refers to the amount of remote data that the fused kernel needs to access during computation on the GPU. Figure 12(a) illustrates the performance variations of the fused kernel compared to the naive implementation (where communication and computation are separate) under variable data locality. As data locality decreases, the time cost of the naive implementation increases linearly, primarily due to increased communication overhead from poor data locality, as shown by Naive AG and Naive GA in Figure 12(a). However, the fused kernel maintains a relatively stable time

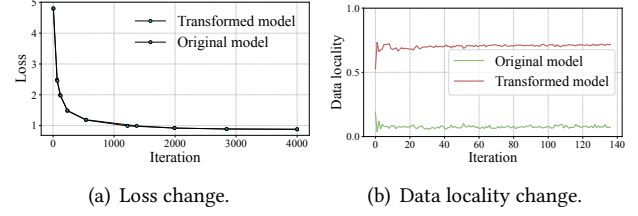


Figure 13. Impact of resource-efficient expert reassignment.

overhead when data locality is above 0.7. This stability is attributed to the inter- and intra-threadblock overlapping mechanisms in our kernel, which effectively hide the communication overhead of remote data accessing. Nevertheless, as data locality further decreases, the performance of the integrated kernel also gradually declines.

Figure 12(b) shows the results of evaluating the fused kernel’s performance by varying the number of experts. As the number of experts in the MoE layer increases, both the communication and computation overhead of the naive implementation rise. With the input data size remaining constant, the increase of time cost is primarily due to each expert needing to perform separate communication and computation. This results in multiple launches of communication and computation operations, with each operation handling smaller amounts of data, thereby reducing efficiency. In contrast, the fused kernel maintains stable performance because it hides communication overhead through overlapping mechanisms and consolidates the computations of all experts within the fused kernel, thus reducing computation overhead. This ensures that the fused kernel remains efficient even when the number of experts changes.

4.5 Analysis of the Expert Reassignment

We examine the effectiveness of transformed MoE layers. We apply the transformation discussed in Section 3.2.1 to all MoE layers in the MoE-Transformer-xl model and train both the transformed and original models using the enwik8 dataset [13] on the *pinnacle* server. Figure 13(a) illustrates the loss values for both models throughout the training process. It is evident that the loss values for the transformed model and the original model are nearly identical during training. This indicates that the Gate network of the transformed model continues to effectively capture token routing information, thus not affecting the model convergence.

Figure 13(b) shows the changes in data locality after implementing the resource-efficient expert reassignment during training. Initially, data locality fluctuates significantly due to the instability of popular experts. However, as training progresses, the resource-efficient expert reassignment algorithm notably improves data locality on the local GPU, stabilizing it at around 70%, compared to less than 20% for the implementation without expert reassignment. This improvement

creates favorable conditions for optimizing the Fused kernel, thereby enhancing training efficiency.

5 Related works

With the successful application of sparsely activated MoE models in Transformer architectures [9, 16, 17, 28], research is increasingly focused on enhancing MoE training performance through algorithmic and system optimizations.

Algorithmic Optimization: Current efforts for MoE models primarily address performance issues related to load imbalance. Gshard [17] and other studies [12, 44] have introduced the concept of expert capacity, which balances the workload by limiting the number of tokens each expert can process. These methods may lead to token drops or sub-optimal expert assignments, ultimately impairing model performance. CCFuser enhances training efficiency by reassigning experts to balance GPU loads without altering the token routing process.

System Optimization: Distributed MoE training is hindered by All2All communication. Therefore, some optimization methods focus on two aspects to improve All2All performance: optimizing the All2All primitive to utilize higher bandwidth [14, 35, 39], and compressing the data to be transmitted to reduce communication time [43]. Additionally, other approaches [4, 12, 19, 21, 29, 30, 41] aim to hide the cost of All2All communication by overlapping it with other sub-tasks to improve training efficiency. In contrast, CCFuser eliminates the reliance on All2All communication by enabling the concurrent processing of local and remote data within the kernel and hiding remote data access latency through inter- and intra-threadblock overlapping.

Communication scheduling techniques have been extensively employed to accelerate distributed training [3, 15, 31, 34, 37]. Centauri [3] optimizes collective communication, such as AllReduce and Allgather in distributed training by partitioning communication across multiple dimensions. It schedules tasks from three perspectives: operation, layer, and model, which enables the overlapping of communication latency and enhances hardware utilization. Additionally, various approaches [7, 8, 14, 20, 22, 26, 32, 40] have implemented different parallel strategies, including expert parallelism, pipeline parallelism, and model parallelism, to accelerate the distributed training on large-scale GPUs. These system optimizations are orthogonal to our work and can be integrated with CCFuser to further boost training efficiency.

6 CONCLUSIONS

CCFuser is a novel framework that accelerates MoE model training by integrating inter-GPU shared memory techniques. It fuses communication and computation into CUDA kernels, eliminating the cross-GPU synchronization overhead typical in existing frameworks. It employs inter- and intra-threadblock overlapping within kernels, hiding remote data

access overhead and boosting GPU utilization. CCFuser also features a resource-efficient expert reassignment strategy that improves GPU data locality and workload balance, thereby reducing the overhead associated with expert reassignment and enhancing training performance. Experimental results show CCFuser substantially speeds up MoE training.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (62341410), National Key Research and Development Program of China (2023YFE0205700), Science and Technology Development Fund, Macao S.A.R (FDCT) project (0078/2023/AMJ).

References

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 1877–1901, 2020.
- [2] Junbum Cha, Wooyoung Kang, Jonghwan Mun, and Byungseok Roh. Honeybee: Locality-enhanced projector for multimodal llm. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13817–13827, 2024.
- [3] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, and Chao Yang. Centauri: Enabling efficient scheduling for communication-computation overlap in large model training via communication partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, page 178–191, 2024.
- [4] Zihao Chen, Chen Xu, Weining Qian, and Aoying Zhou. Elastic averaging for efficient pipelined dnn training. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*, page 380–391, 2023.
- [5] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 2978–2988, 2019.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (ACL)*, pages 4171–4186, 2019.
- [7] Jiangsu Du, Jinhui Wei, Jiazhi Jiang, Shenggan Cheng, Dan Huang, Zhiguang Chen, and Yutong Lu. Liger: Interleaving intra- and inter-operator parallelism for distributed large model inference. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*, page 42–54, 2024.
- [8] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, et al. Dapple: A pipelined data parallel approach for training large models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 431–445, 2021.
- [9] Jiarui Fang, Yang Yu, Chengduo Zhao, and Jie Zhou. Turbo Transformers: an efficient gpu serving system for transformer models. In

- Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 389–402, 2021.
- [10] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research (JMLR)*, pages 1–39, 2022.
 - [11] Jiaao He, Jiezhong Qiu, Aohan Zeng, Zhilin Yang, Jidong Zhai, and Jie Tang. Fastmoe: A fast mixture-of-expert training system. *arXiv preprint arXiv:2103.13262*, 2021.
 - [12] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. Fastermoe: modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 120–134, 2022.
 - [13] Marcus Hutter. the human knowledge compression contest. <http://prize.hutter1.net/>, 2012.
 - [14] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, et al. Tutel: Adaptive mixture-of-experts at scale. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023.
 - [15] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. Breaking the computation and communication abstraction barrier in distributed machine learning workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, page 402–416, 2022.
 - [16] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, L  lio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Th  ophile Gerv  t, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mixtral of experts. *CoRR*, 2024.
 - [17] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. {GS}hard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021.
 - [18] Mike Lewis, Shruti Bhosale, Tim Dettmers, Naman Goyal, and Luke Zettlemoyer. Base layers: Simplifying training of large, sparse models. In *International Conference on Machine Learning (ICML)*, pages 6265–6274, 2021.
 - [19] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. Accelerating distributed {MoE} training and inference with lina. In *2023 USENIX Annual Technical Conference (ATC)*, pages 945–959, 2023.
 - [20] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 663–679, 2023.
 - [21] Juncai Liu, Jessie Hui Wang, and Yimin Jiang. Janus: A unified distributed training framework for sparse mixture-of-experts models. In *Proceedings of the ACM SIGCOMM 2023 Conference (SIGCOMM)*, pages 486–498, 2023.
 - [22] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 1–15, 2021.
 - [23] Xiaonan Nie, Xupeng Miao, Zilong Wang, Zichao Yang, Jilong Xue, Lingxiao Ma, Gang Cao, and Bin Cui. Flexmoe: Scaling large-scale sparse pre-trained model training via dynamic device placement. In *Proceedings of the ACM on Management of Data (PACMOD)*, pages 1–19, 2023.
 - [24] Nvidia. Nvidia collective communication library (nccl). <https://developer.nvidia.com/nccl>, 2024.
 - [25] Nvidia. Nvshmem communication library. <https://developer.nvidia.com/nvshmem>, 2024.
 - [26] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (SIGKDD)*, pages 3505–3506, 2020.
 - [27] Carlos Riquelme, Joan Puigcerver, Basil Mustafa, Maxim Neumann, Rodolphe Jenatton, Andr   Susano Pinto, Daniel Keysers, and Neil Houlsby. Scaling vision with sparse mixture of experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 8583–8595, 2021.
 - [28] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.
 - [29] Shaohuai Shi, Xinglin Pan, Xiaowen Chu, and Bo Li. Pipemoe: Accelerating mixture-of-experts through adaptive pipelining. In *IEEE INFOCOM 2023-IEEE Conference on Computer Communications (INFOCOM)*, pages 1–10, 2023.
 - [30] Shaohuai Shi, Xinglin Pan, Qiang Wang, Chengjian Liu, Xiaozhe Ren, Zhongzhe Hu, Yu Yang, Bo Li, and Xiaowen Chu. Schemoe: An extensible mixture-of-experts distributed training system with tasks scheduling. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*, pages 236–249, 2024.
 - [31] Jaeyong Song, Jinkyu Yim, Jaewon Jung, Hongsun Jang, Hyung-Jin Kim, Youngsok Kim, and Jinho Lee. Optimus-cc: Efficient large nlp model training with 3d parallelism aware communication compression. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, page 560–573, 2023.
 - [32] Zhenbo Sun, Huanqi Cao, Yuanwei Wang, Guanyu Feng, Shengqi Chen, Haojie Wang, and Wenguang Chen. Adapipe: Optimizing pipeline parallelism with adaptive recomputation and partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, page 86–100, 2024.
 - [33] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aur  lien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, 2023.
 - [34] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, Sameer Kumar, Tongfei Guo, Yuanzhong Xu, and Zongwei Zhou. Overlap communication with dependent computation via decomposition in large deep learning models. In *Proceedings of*

- the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), page 93–106, 2022.
- [35] Weihua Wang, Yaqi Xia, Donglin Yang, Xiaobo Zhou, and Dazhao Cheng. Accelerating distributed dlm training with optimized tt decomposition and micro-batching. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, 2024.
 - [36] Wenhui Wang, Jifeng Dai, Zhe Chen, Zhenhang Huang, Zhiqi Li, Xizhou Zhu, Xiaoqi Hu, Tong Lu, Lewei Lu, Hongsheng Li, Xiaogang Wang, and Yu Qiao. Internimage: Exploring large-scale vision foundation models with deformable convolutions. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14408–14419, 2023.
 - [37] Yuke Wang, Boyuan Feng, Zheng Wang, Tong Geng, Kevin Barker, Ang Li, and Yufei Ding. MGG: Accelerating graph neural networks with Fine-Grained Intra-Kernel Communication-Computation pipelining on Multi-GPU platforms. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 779–795, 2023.
 - [38] Yaqi Xia, Donglin Yang, Xiaobo Zhou, and Dazhao Cheng. Scaling new heights: Transformative cross-gpu sampling for training billion-edge graphs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, 2024.
 - [39] Yaqi Xia, Zheng Zhang, Hulin Wang, Donglin Yang, Xiaobo Zhou, and Dazhao Cheng. Redundancy-free high-performance dynamic gnn training with hierarchical pipeline parallelism. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*, page 17–30, 2023.
 - [40] Tailing Yuan, Yuliang Liu, Xucheng Ye, Shenglong Zhang, Jianchao Tan, Bin Chen, Chengru Song, and Di Zhang. Accelerating the training of large language models using efficient activation rematerialization and optimal hybrid parallelism. In *2024 USENIX Annual Technical Conference (ATC)*, pages 545–561, 2024.
 - [41] Mingshu Zhai, Jiaao He, Zixuan Ma, Zan Zong, Runqing Zhang, and Jidong Zhai. {SmartMoE}: Efficiently training {Sparsely-Activated} models through combining offline and online parallelization. In *2023 USENIX Annual Technical Conference (ATC)*, pages 961–975, 2023.
 - [42] Zheng Zhang, Donglin Yang, Yaqi Xia, Liang Ding, Dacheng Tao, Xiaobo Zhou, and Dazhao Cheng. Mpipemoe: Memory efficient moe for pre-trained models with adaptive pipeline parallelism. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 167–177, 2023.
 - [43] Qinghua Zhou, Pouya Kousha, Quentin Anthony, Kawthar Shafie Khorrassani, Aamir Shafi, Hari Subramoni, and Dhabaleswar K Panda. Accelerating mpi all-to-all communication with online compression on modern gpu clusters. In *International Conference on High Performance Computing*, pages 3–25, 2022.
 - [44] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 7103–7114, 2022.